Technical Report
Embedded System Project: PC FAN speed controller
Author: Martins Osemudiamen Okoh.
TU Dublin ID: A0004717

Aim: Design and implementation of a PC fan speed controller using STM32L432KC microcontroller.

Objective:
1. Schematic design of the PC fan speed controller.
2. Software implementation.
3. Hardware implementation.
4. Testing / Debugging.
5. Documentation.

Design:

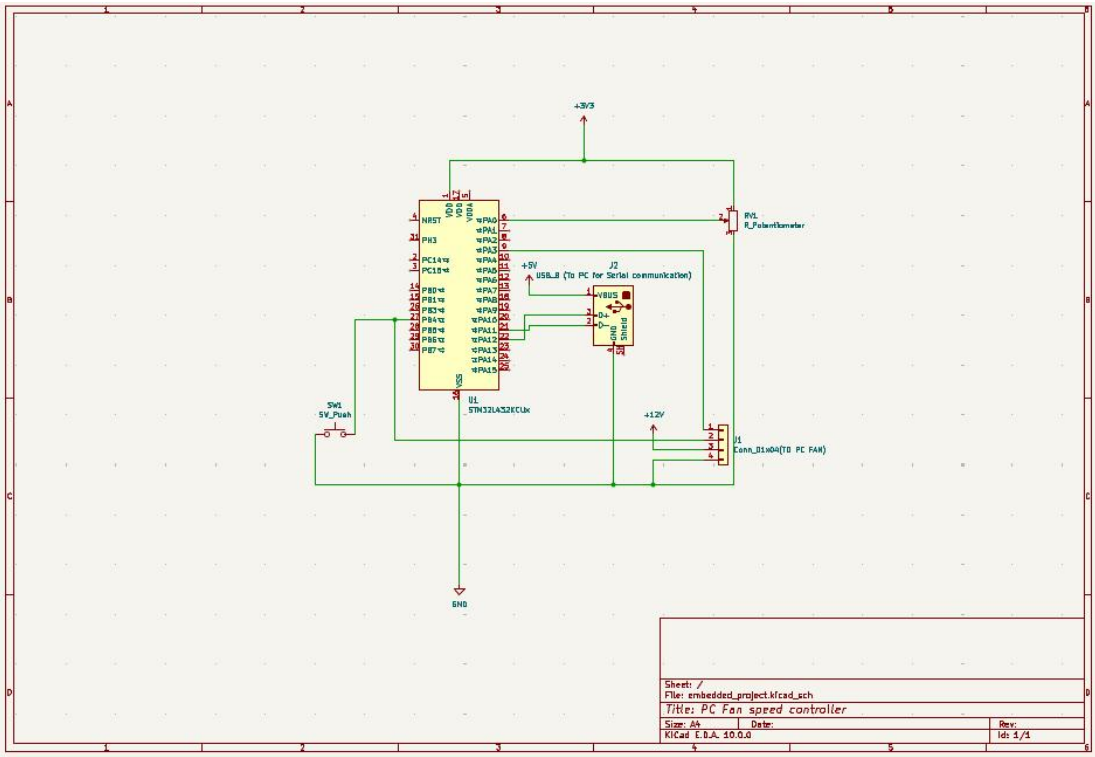

Figure 1: Ciricuit Design of PC fan controller using KICad 10.0.

In the Figure 1 above, the design comprises of a push button, potentiometer, PC fan 4 wire connector, a USB cable component for serial communication, voltage supplies of 3.3 and 12 voltages and a ground.

Source Code:

```c
#include <stm32l432xx.h>
#include <stdint.h>
void setup(void);
void delay(volatile uint32_t dly);
void enablePullUp(GPIO_TypeDef *Port, uint32_t BitNumber);
void pinMode(GPIO_TypeDef *Port, uint32_t BitNumber, uint32_t Mode);
void initADC(void);
int readADC(int chan);
void initTimer2(void);
void setTimer2Duty(int duty);
void initClocks(void);
void delay_ms(unsigned dly);

// added from serial file
#include <eeng1030_lib.h>
#include <stdio.h>
#include  <errno.h>
#include  <sys/unistd.h> // STDOUT_FILENO, STDERR_FILENO

void initSerial(uint32_t baudrate);
void eputc(char c);
```

Figure 2: Declaration of functions and addition of necessary files and libraries.

```c
// define needed constants
#define pulse_per_rev 2

int vin, speed_hz, speed_rpm;
volatile unsigned pulse_count, milliseconds;
```

Figure 3: Declaration of global variables.

```c
int main()
{
    setup();
    SysTick->LOAD = 80000-1; // Systick clock = 80MHz. 80000000/80000=1000
    SysTick->CTRL = 7; // enable systick counter and its interrupts
    SysTick->VAL = 10; // start from a low number so we don't wait for ages for first interrupt

    while(1)
    {
        vin = readADC(5);
        setTimer2Duty(vin);

        pulse_count = 0;

        delay_ms(1000);

        // evaluate speed
        speed_hz = (pulse_count / pulse_per_rev);
        speed_rpm = (speed_hz * 60);
        // print fan speed
        printf("For ADC: %d, the Fan speed (rpm) is %d\r\n", vin, speed_rpm);

    }
}
```

Figure 4: Main function of the software implementation.

As shown in Figure 4 above, the sequence flow of the code starts with the setup function and the clock counter with interrupt. A continuous loop is initiated, which reads the resistor value from the potentiometer that controls the PC speed. A variable, pulse_count, is firstly initialized to 0 and a milliseconds delay of 1000 is then performed. This delay ensures the pulse_count is captured accurately and its value is used to estimate the PC fan speed in revolutions per minute. The speed with the potentiometer level is read via serial communication unto the terminal display using the equation (1) below.

$$RPM = (pulse_count / pulses_per_rev) * 60 \quad \ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots (1)$$

```c
void setup(void)
{
    initClocks();
    RCC->AHB2ENR |= (1 << 0) | (1 << 1); // turn on GPIOA and GPIOB

    pinMode(GPIOB,3,1); // digital output
    pinMode(GPIOB,4,0); // digital input
    enablePullUp(GPIOB,4); // pull-up for button
    pinMode(GPIOA,0,3);  // analog input

    pinMode(GPIOA,3,2);  // alternative function mode
    GPIOA->AFR[0] &= ~(3 << (2*3));    // Clear out old alternative function  bits
    //GPIOA->AFR[0] &= ~(0xF << (3*4));
    GPIOA->AFR[0] |= 1 << (4*3);    // Select alternative function 1

    // serial communication
    initADC();
    initSerial(9600);
    initTimer2();

    // interrupt for counting pulses
    RCC->APB2ENR |= (1 << 0); // enable SYSCFG
    SYSCFG->EXTICR[1] &= ~(7 << 0); // clear perhaps previously set bits
    SYSCFG->EXTICR[1] |= (1 << 0);  // map EXTI2 interrupt to PB4
    EXTI->FTSR1 |= (1 << 4); // select falling edge trigger for PB4 input
    EXTI->IMR1 |= (1 << 4);  // enable PB4 interrupt
    NVIC->ISER[0] |= (1 << 10); // IRQ 10 maps to EXTI4
    __enable_irq();

}
```

Figure 5: Function setup.

The setup function shown in Figure 5 above includes pin definitions, functions for initialization of clocks, serial communication to display terminals and interrupt for pulse counting. The interrupt is mapped to PB4 pin which is enabled and select the falling edge of the tachometer signal / speed signal from the fan feed to the PB4 pin. GPIO pins are set (PA3 , PA0,PB4) as inputs and outputs.

```c
void initADC()
{
    // initialize the ADC
    RCC->AHB2ENR |= (1 << 13); // enable the ADC
    RCC->CCIPR |= (1 << 29) | (1 << 28); // select system clock for ADC
    // set ADC clock = HCLK and turn on the voltage reference
    ADC1_COMMON->CCR = ((0b01) << 16) + (1 << 22) ;
    // start ADC calibration
    ADC1->CR=(1 << 28); // turn on the ADC voltage regulator and disable the ADC
    // wait for voltage regulator to stabilize
    //(20 microseconds according to the datasheet).
    //This gives about 180microseconds
    delay(100);
    ADC1->CR |= (1<< 31);
    while(ADC1->CR & (1 << 31)); // wait for calibration to finish.
    ADC1->CFGR = (1 << 31); // disable injection
    ADC1_COMMON->CCR |= (0x0f << 18);
}
```

Figure 6: Analogy to digital conversion initialization function.

```c
int readADC(int chan)
{

    ADC1->SQR1 |= (chan << 6);
    ADC1->ISR = (1 << 3); // clear EOS flag
    ADC1->CR |= (1 << 0); // enable the ADC
    while ( (ADC1->ISR & (1 <<0))==0); // wait for ADC to be ready
    ADC1->CR |= (1 << 2); // start conversion
    while ( (ADC1->ISR & (1 <<3))==0); // wait for conversion to finish
    return ADC1->DR; // return the result
    ADC1->CR = 0;
}
```

Figure 7: Analogy to digital conversion reading function.

```c
void delay(volatile uint32_t dly)
{
    while(dly--);
}
```

Figure 8: Function of installing delay.

```c
void enablePullUp(GPIO_TypeDef *Port, uint32_t BitNumber)
{
    Port->PUPDR = Port->PUPDR &~(3u << BitNumber*2); // clear pull-up resistor bits
    Port->PUPDR = Port->PUPDR | (1u << BitNumber*2); // set pull-up bit
}
```

Figure 9: Function to enable pull-up resistor.

```c
void pinMode(GPIO_TypeDef *Port, uint32_t BitNumber, uint32_t Mode)
{
    /*
        Modes : 00 = input
                01 = output
                10 = special function
                11 = analog mode
    */
    uint32_t mode_value = Port->MODER;
    Mode = Mode << (2 * BitNumber);
    mode_value = mode_value & ~(3u << (BitNumber * 2));
    mode_value = mode_value | Mode;
    Port->MODER = mode_value;
}
```

Figure 10: Function for PIN mode selection.

```c
void initTimer2(void)
{
    RCC->APB1ENR1 |= (1 << 0); // enable Timer 2
    TIM2->CR1 = 0;
    TIM2->CCMR2 = (0b110 << 12) + (1 << 11)+(1 << 10);
    TIM2->CCER |= (1 << 12);
    TIM2->ARR = 2000-1;
    TIM2->CCR4 = 500;
    TIM2->EGR |= (1 << 0);
    TIM2->CR1 = (1 << 7);
    TIM2->CR1 |= (1 << 0);
}
```

Figure 11: Function for initializing timer.

```c
void setTimer2Duty(int duty)
{
    int arrvalue=(duty*TIM2->ARR)/4095;
    TIM2->CCR4=arrvalue;
}
```

Figure 12: Function for setting timer for pulse width modulation duty cycle.

In Figure 12 above, the duty setting of the potentiometer from 0 to 4095 is done.

```c
void initClocks()
{
    // Initialize the clock system to a higher speed.
    // At boot time, the clock is derived from the MSI clock
    // which defaults to 4MHz.  Will set it to 80MHz
    // See chapter 6 of the reference manual (RM0393)
        RCC->CR &= ~(1 << 24); // Make sure PLL is off

        RCC->PLLCFGR = (1 << 25) + (1 << 24) + (1 << 22) + (1 << 21) + (1 << 17) + (80 << 8) + (1 << 0);
        RCC->CR |= (1 << 24); // Turn PLL on
        while( (RCC->CR & (1 << 25))== 0); // Wait for PLL to be ready
    // configure flash for 4 wait states (required at 80MHz)
        FLASH->ACR &= ~((1 << 2)+ (1 << 1) + (1 << 0));
        FLASH->ACR |= (1 << 2);
        RCC->CFGR |= (1 << 1)+(1 << 0); // Select PLL as system clock
}
```

Figure 13: Function for initializing clock

```c
// added from serial communication file
void initSerial(uint32_t baudrate)
{
    RCC->AHB2ENR |= (1 << 0); // make sure GPIOA is turned on
    pinMode(GPIOA,2,2); // alternate function mode for PA2
    selectAlternateFunction(GPIOA,2,7); // AF7 = USART2 TX
    pinMode(GPIOA,15,2);
    selectAlternateFunction(GPIOA,15,3);
    RCC->APB1ENR1 |= (1 << 17); // turn on USART2

    const uint32_t CLOCK_SPEED=80000000;
    uint32_t BaudRateDivisor;

    BaudRateDivisor = CLOCK_SPEED/baudrate;
    USART2->CR1 = 0;
    USART2->CR2 = 0;
    USART2->CR3 = (1 << 12); // disable over-run errors
    USART2->BRR = BaudRateDivisor;
    USART2->CR1 =  (1 << 3);  // enable the transmitter and receiver
    USART2->CR1 |=  (1 << 2);  // enable the transmitter and receiver
    USART2->CR1 |= (1 << 0);
}
```

Figure 14: Function of initializing serial communication and pin definition.

Figure 14 above shows how the PA2 and AF7 are set internally for transmitting and receiving of data via the USB serial cable.

```c
int _write(int file, char *data, int len)
{
    if ((file != STDOUT_FILENO) && (file != STDERR_FILENO))
    {
        errno = EBADF;
        return -1;
    }
    while(len--)
    {
        eputc(*data);
        data++;
    }
    return 0;
}
```

Figure 15: Function for writing via serial communication.

```c
void eputc(char c)
{
    while( (USART2->ISR & (1 << 6))==0); // wait for ongoing transmission to finish
    USART2->TDR=c;
}
```

Figure 16: Function for write via serial communication to terminal display.

```c
void EXTI4_IRQHandler()
{
    //PIOB->BSRR = (1 << 3); // set PB3 to turn on LED
    //GPIOB->IDR |= (1 << 4);
    EXTI->PR1 = (1 << 4);   // clear interrupt pending flag
    pulse_count++; //counting the pulse

}
```

Figure 17: Function to handle interrupt to counting the pulse for the speed estimation of the PC fan.

The Figure 17 above show the using of PB4 for interrupt handling and the pulse_count variable increment performed to track the speed signal.

```c
void SysTick_Handler(void)
{
   milliseconds++;
}
void delay_ms(unsigned dly){
    unsigned end = milliseconds + dly;
    while(milliseconds != end);
}
```

Figure 18: Function for milliseconds counting and delays.

The Figure 18 above show the using creation of a millisecond delay to enable precise tracking of the speed signal which enables the pulse_count increment.

Testing:

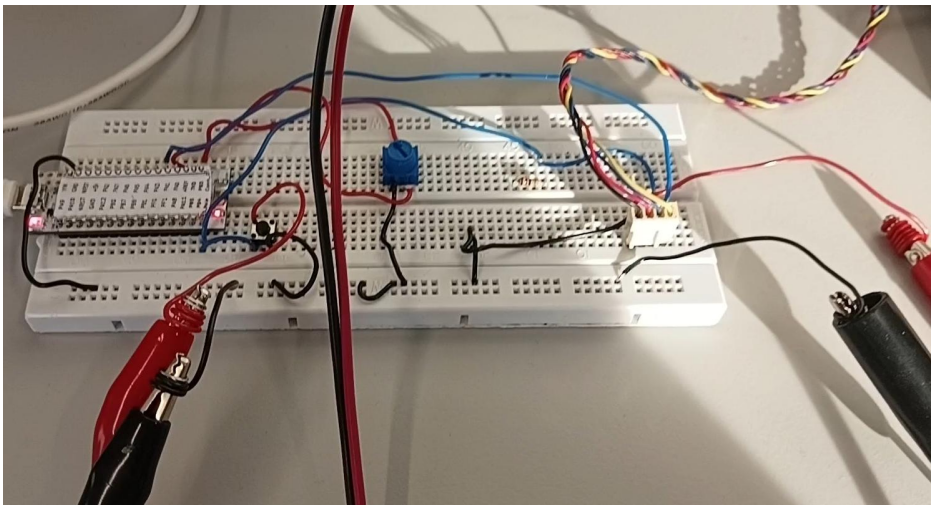

Figure 19: Hardware implementation of bread board.

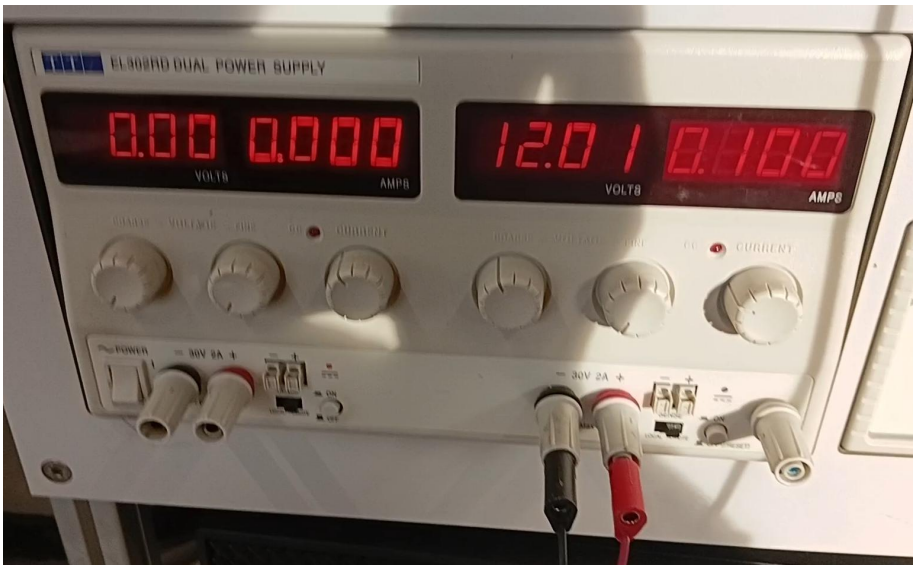

Figure 20: Power supply.

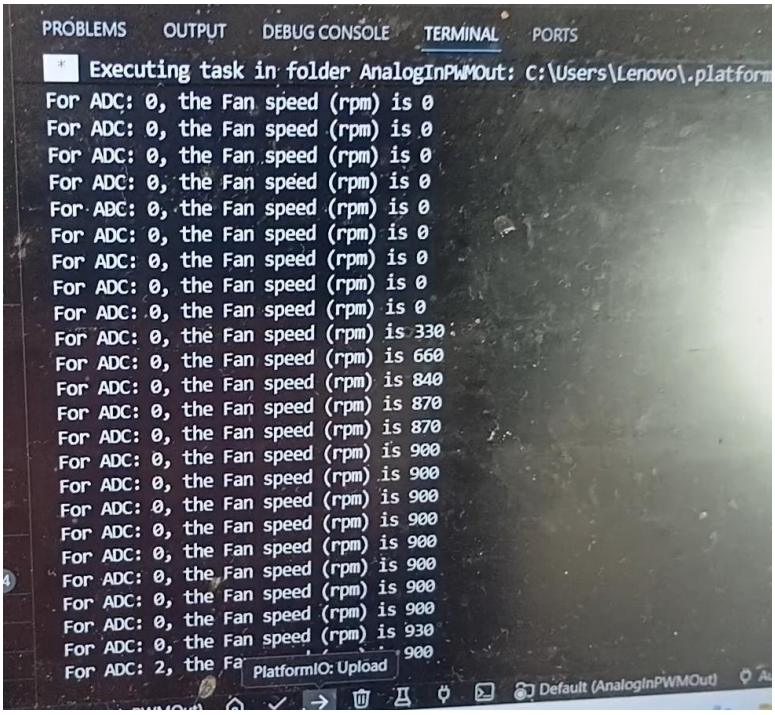

Figure 21: Output result 1 on terminal.

The result in Figure 21 shows the speed of max of 930 revolution per minute when the potentiometer is at minimum level.

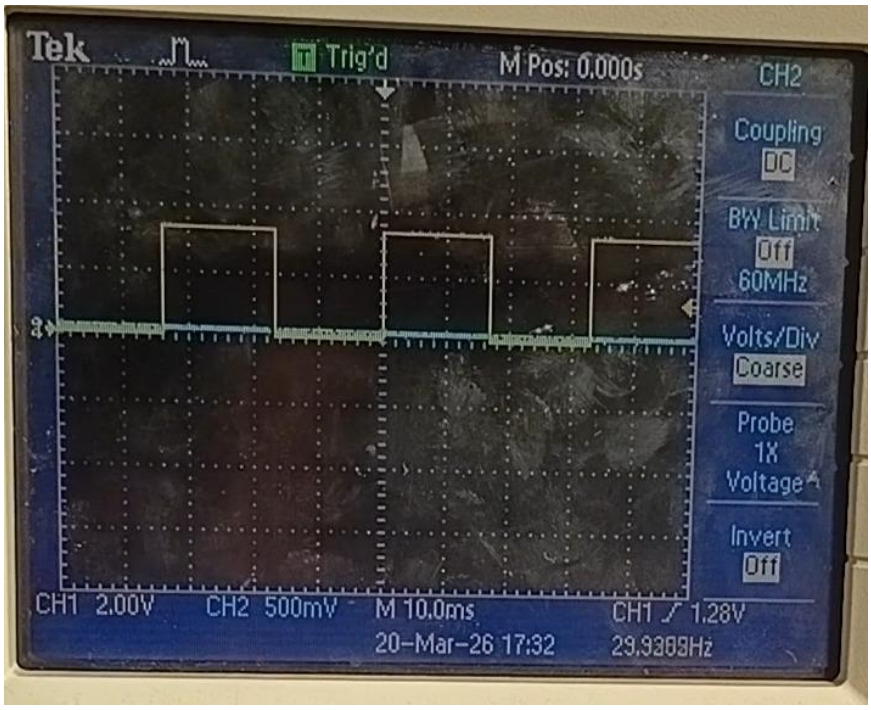

Figure 22: Fan output speed signal 1 viewed on oscilloscope.

The result in Figure 22 shows the output speed signal from the PC fan with a frequency of 29.9Hz.

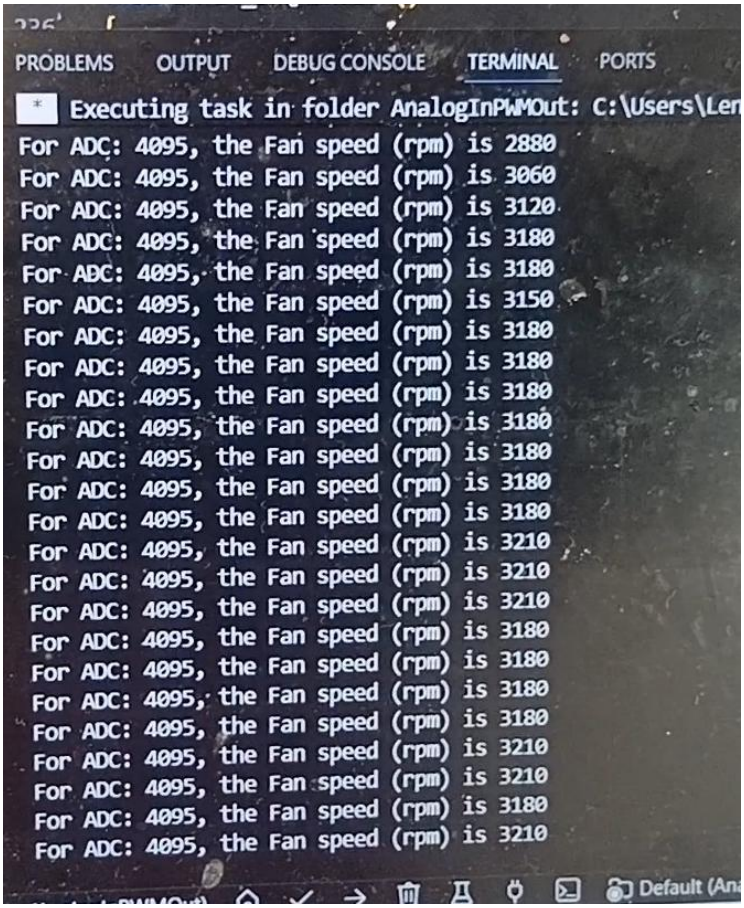

Figure 23: Output result 2 on terminal.

The result in Figure 23 shows the speed of max of 3210 revolution per minute when the potentiometer is at maximum level.

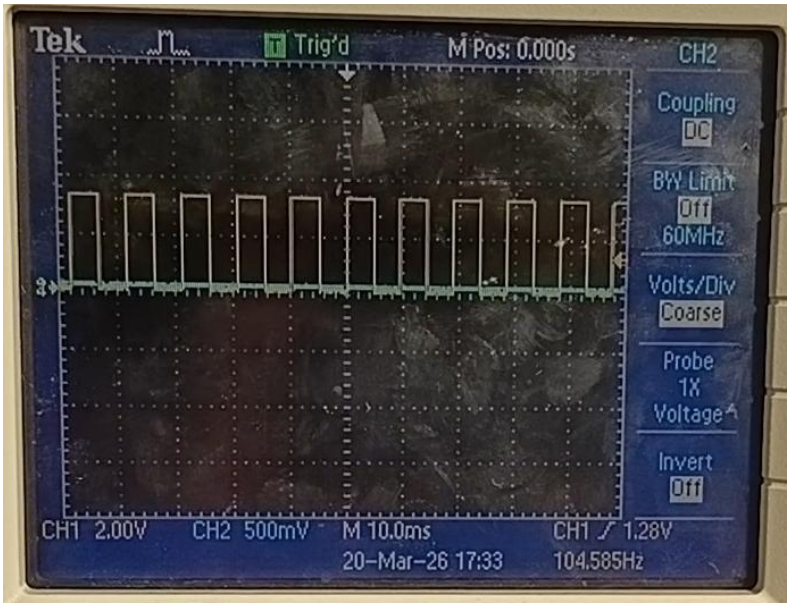

Figure 24: Fan output speed signal 2 viewed on oscilloscope.

The result in Figure 22 shows the output speed signal from the PC fan with a frequency of 104.58Hz.

Conclusion:

The project aim and objective were observed to have been achieved. The circuit design, software and hardware implementation were effective performed. The fan speed is able to be controlled by a potentiometer and the serial communication is able to read the speed value unto the terminal. An oscilloscope was used to perform hardware debugging while break points were used for software code debugging.

References:

1) M. S. Waghare, D. R. Tutakne, A. N. Deshmukh, and M. M. Mardikar, “PWM controlled high power factor single phase fan regulator,” in Proc. 3rd Int. Conf. Electronics, Communication and Aerospace Technology (ICECA), 2019, pp. 59–64, doi: 10.1109/ICECA.2019.8822156.